

Textbook: A-Star

Pathfinding algorithms are designed to find the shortest or most efficient path between two points. They are widely used in fields like robotics, video games, and map navigation. There are several algorithms available, each with its strengths and use cases.

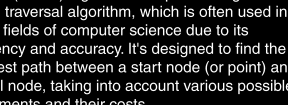
1. **Dijkstra's Algorithm:** Finds the shortest path from a single source to all other nodes in a graph. It's efficient but does not consider the direction of the goal, potentially leading to longer processing times.
2. **Breadth-First Search (BFS):** Explores neighbor nodes first before moving to the next-level neighbors. It's great for finding the shortest path in unweighted graphs but can be inefficient in larger spaces.
3. **Depth-First Search (DFS):** Explores as far as possible along a branch before backtracking. It's not guaranteed to find the shortest path, but it can be useful in scenarios where space is limited.
4. **A-Star (A) Algorithm***: Combines features of Dijkstra's and BFS, using heuristics to improve performance. It's efficient and guarantees the shortest path, making it popular for grid-based pathfinding.

The A-Star (A*) Algorithm

A* is a popular choice for pathfinding thanks to its efficiency and accuracy. It uses heuristics to estimate the distance from the current node to the goal, reducing the number of explored nodes.

A* (A-STAR)

The A* (A-star) algorithm is a pathfinding and graph traversal algorithm, which is often used in many fields of computer science due to its efficiency and accuracy. It's designed to find the shortest path between a start node (or point) and a goal node, taking into account various possible movements and their costs.



Legend:

- Start Node
- End Node
- Optimal Path
- Open set

Components of A*:

1. **G Cost (Actual Cost):** The cost of moving from the start node to a given node along the best known path.
2. **H Cost (Heuristic Cost):** An estimated cost from that node to the goal. Common heuristics include Euclidean distance and Manhattan distance.
3. **F Cost:** The sum of G Cost and H Cost ($F = G + H$). This cost is used to prioritize the node exploration.

Algorithm Steps:

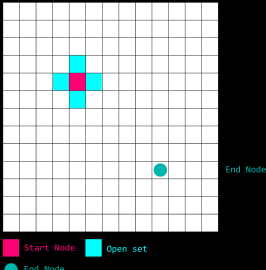
1. **Initialize:** Put the start node in an **open** and **closed list**. The open list will contain the nodes discovered but not evaluated and the closed list will contain the nodes that have already been evaluated.
2. **Loop:**
 - a. Choose the node with the lowest F Cost from the open list.
 - b. Switch it to the closed list (processed nodes).
 - c. For each neighbor of this node:
 - If it's not walkable or in the closed list, ignore it.
 - If it's not on the open list, add it. Calculate G, H, and F costs.
 - If it's in the open list but the new G cost is lower, update its costs and parent.
3. **Goal Reached:** If the goal is added to the closed list or the open list is empty, the algorithm ends.
4. **Path Reconstruction:** Trace the path from the goal node back to the start node using parent references.

A* (A-STAR)

Start by initializing two sets:
the **open set** and the **closed set**.

The open set contains nodes that have been discovered but not evaluated, and the closed set contains nodes that have already been evaluated.

Initially, the open set contains just the start node and the closed set is empty.



End Node

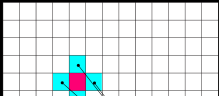
Start Node Open set End Node

A* (A-STAR)

Heuristic Function (h):

A* uses a heuristic function to estimate the cost of the cheapest path from a node to the goal. This heuristic is problem-specific.

A common heuristic for grid-based games is the distance to the end node.



Distance Calculation

- 1. G Cost (Actual Cost):**
 - The G Cost is a measure of the actual cost of the path from the start node to a given node.
 - It represents the exact cost (or distance) traveled from the start node to the current node.
 - In a grid, this could be the sum of the distances of all the moves made (counting each move as a unit step or more if diagonal steps have a higher cost).
 - The G Cost is cumulative; as the algorithm progresses, the G Cost of each node reflects the total cost of getting to that node from the start.
- 2. H Cost (Heuristic Cost):**
 - The H Cost is an estimate of the cost from the current node to the goal node.
 - This is a heuristic measure and not an exact calculation (since the exact cost is unknown until the goal is reached).
 - Common heuristics include the Euclidean distance (straight-line distance) in non-grid environments or the Manhattan distance (sum of the absolute values of the horizontal and vertical differences) in grid-

